

GUIDE MÉTHODOLOGIQUE

Comment créer un site web ou une plateforme — de zéro jusqu'à la fin

Une cartographie en 9 étapes, regroupées en 3 grandes phases : réfléchir avant de coder, construire, puis lancer et faire évoluer. Applicable aussi bien à un site vitrine simple qu'à une plateforme complexe avec plusieurs types d'utilisateurs.

LE PIPELINE COMPLET, VUE D'ENSEMBLE



Ce guide décrit un processus générique. Selon la taille du projet, plusieurs étapes peuvent être menées en parallèle ou fusionnées (par exemple un site vitrine simple n'a pas besoin d'une phase d'architecture aussi poussée qu'une plateforme multi-utilisateurs). L'important est de ne sauter aucune étape sans décision consciente.

PARTIE 1

Avant de coder

Le temps passé ici évite le plus de refonte plus tard : ce sont des décisions, pas encore du code.

1 Cadrage

Porté par : porteur du projet / chef de produit

Avant toute ligne de code, il faut répondre à trois questions : quel problème le site ou la plateforme résout-il, pour qui, et avec quelles limites ? Un projet mal cadré coûte cher à corriger une fois développé.

ÉTAPES CONCRÈTES

- Définir l'objectif principal en une phrase
- Identifier les types d'utilisateurs (personas) et leurs besoins
- Étudier 2-3 sites/plateformes concurrents ou similaires
- Définir le périmètre du premier lancement (MVP) vs. ce qui viendra après
- Lister les contraintes : budget, délai, obligations légales (données personnelles, secteur réglementé)

LIVRABLES

Note de cadrage

Personas

Liste de fonctionnalités priorisées

Piège à éviter

Vouloir tout construire dès le premier lancement. Un périmètre trop large retarde la mise en ligne et complique chaque étape suivante.

2 Spécifications fonctionnelles

Porté par : chef de produit, avec les futurs utilisateurs si possible

Traduire les objectifs du cadrage en fonctionnalités précises et vérifiables : qui peut faire quoi, dans quel ordre, avec quelles règles. C'est le document de référence que développeurs et designers utiliseront pour ne pas réinterpréter le besoin chacun à sa façon.

ÉTAPES CONCRÈTES

- Rédiger les parcours utilisateurs sous forme de user stories (« en tant que X, je veux Y, afin de Z »)
- Définir les rôles et permissions (ex : visiteur, utilisateur inscrit, administrateur)
- Modéliser les processus métier à plusieurs étapes (ex : un parcours de commande, de candidature, d'inscription)
- Prioriser avec une méthode simple (Must have / Should have / Could have / Won't have)

LIVRABLES

Cahier des charges fonctionnel

Wireframes basse fidélité

Matrice des rôles

3 Architecture technique

Porté par : développeur / architecte technique

Traduire les spécifications en décisions techniques : quelle stack, comment les données sont organisées, comment la sécurité et les accès sont gérés. Ces choix sont coûteux à changer une fois le développement avancé — mieux vaut les documenter avant de coder.

ÉTAPES CONCRÈTES

- Choisir la stack (langage, framework, base de données, hébergement)
- Modéliser les données : quelles entités, quelles relations entre elles
- Définir l'architecture générale (application unique, API + frontend séparés, etc.)
- Prévoir l'authentification, les rôles et le contrôle d'accès dès la conception
- Prévoir plusieurs environnements (développement, test, production)

LIVRABLES

Schéma de données

Choix techniques documentés

Schéma d'architecture

Conseil

Choisir une stack que l'équipe maîtrise déjà, ou pour laquelle des ressources d'apprentissage solides existent, plutôt que la technologie la plus récente.

4 Design UX/UI

Porté par : designer (ou le développeur, sur un petit projet)

Donner une forme concrète aux parcours définis en spécifications : ce que l'utilisateur voit, dans quel ordre, avec quelle hiérarchie visuelle. Un design system cohérent (couleurs, typographie, composants) accélère ensuite tout le développement.

ÉTAPES CONCRÈTES

- Affiner les wireframes en maquettes haute-fidélité, écran par écran
- Définir un design system : palette de couleurs, typographies, espacements, composants réutilisables (boutons, formulaires, cartes)
- Vérifier l'accessibilité (contraste, tailles de police, navigation clavier)
- Tester le prototype auprès de quelques utilisateurs cibles si possible

LIVRABLES

Maquettes haute-fidélité

Design system

Prototype cliquable

PARTIE 2

Construire

Le code est écrit, testé, et vérifié avant de partir en production.

5 Développement

Porté par : développeur(s)

Construire l'application par petites itérations vérifiables plutôt qu'en un seul bloc : chaque fonctionnalité livrée doit pouvoir être testée de bout en bout avant de passer à la suivante.

ÉTAPES CONCRÈTES

- Initialiser le projet : dépôt de code, structure, environnements de développement
- Construire la base de données (modèles, migrations) à partir du schéma défini en architecture
- Développer le backend : logique métier, API, règles d'accès par rôle
- Développer le frontend : interfaces à partir des maquettes, branchées sur l'API
- Intégrer les services tiers nécessaires (envoi d'e-mails, paiement, connexion via un autre service)
- Avancer fonctionnalité par fonctionnalité, en suivant l'ordre de priorité défini en spécifications

LIVRABLES

Code source versionné

Application fonctionnelle en environnement de développement

Porté par : développeur(s), idéalement avec un regard extérieur

Vérifier que ce qui a été construit correspond bien aux spécifications, et qu'une modification future ne casse pas silencieusement une fonctionnalité existante.

ÉTAPES CONCRÈTES

- Tests unitaires sur la logique métier critique (calculs, règles, validations)
- Tests de bout en bout sur les parcours utilisateurs principaux (inscription, action clé, paiement...)
- Tests manuels des cas limites (formulaire vide, fichier trop lourd, accès non autorisé)
- Vérifications de sécurité de base (contrôle d'accès par rôle, validation des données envoyées par l'utilisateur)
- Automatiser ces tests pour qu'ils tournent à chaque modification du code (intégration continue)

LIVRABLES

Suite de tests automatisés

Rapport de bugs corrigés

Piège à éviter

Se fier uniquement à des vérifications manuelles ponctuelles : sans tests automatisés, chaque nouvelle fonctionnalité risque de casser silencieusement une fonctionnalité existante.

PARTIE 3

Lancer et faire vivre

La mise en ligne n'est pas la fin du projet, mais le début de son exploitation réelle.

7

Déploiement

Porté par : développeur(s)

Faire passer l'application de l'environnement de développement à un environnement accessible publiquement, de façon reproductible et sans perte de données.

ÉTAPES CONCRÈTES

- Choisir un hébergement adapté à la stack et au volume attendu
- Configurer le nom de domaine et le certificat de sécurité (HTTPS)
- Renseigner les variables d'environnement de production (clés, secrets) séparément du code
- Appliquer les migrations de base de données en production
- Mettre en place des sauvegardes automatisées de la base de données
- Automatiser les déploiements suivants (intégration/déploiement continu)

LIVRABLES

Application en ligne

Procédure de déploiement documentée

8

Lancement et conformité

Porté par : porteur du projet, avec un conseil juridique si besoin

Un site accessible au public engage des obligations légales, quelle que soit sa taille : mentions légales, conditions d'utilisation, traitement des données personnelles.

ÉTAPES CONCRÈTES

- Rédiger les mentions légales, CGU, et CGV si des paiements sont proposés
- Rédiger une politique de confidentialité si des données personnelles sont collectées, avec les durées de conservation
- Vérifier les obligations de déclaration auprès de l'autorité de protection des données du pays concerné
- Mettre en place un minimum de référencement (SEO) et d'outils de suivi d'audience

LIVRABLES

Pages légales publiées

Conformité vérifiée

Porté par : toute l'équipe, en continu

La mise en ligne marque le début d'un cycle continu : surveiller que tout fonctionne, écouter les retours des utilisateurs, et faire évoluer le produit. Chaque nouvelle fonctionnalité importante repasse par une mini-version des étapes 2 à 7.

ÉTAPES CONCRÈTES

- Mettre en place une supervision de base (disponibilité, erreurs serveur)
- Organiser un canal de support ou de retour utilisateur
- Suivre des indicateurs d'usage pour comprendre ce qui fonctionne ou non
- Prioriser et planifier les évolutions futures à partir de ces retours

LIVRABLES

Tableau de supervision

Feuille de route d'évolution